

Lab: Forearm movements and PPO

Julien Gori

In the previous lab, we were able to solve Bellman's equation assuming linear dynamics as well as quadratic costs. These assumptions are strong, and we might want to relax those for a more realistic model. In this lab, we look at a model-free Reinforcement Learning algorithm known as PPO to solve our optimization problem.

We are going to use a library called `stable-baselines3` with an off-the-shelf implementation of PPO. To be compatible with stable baselines, we will comply with the Gym API, which is a standard for implementing RL environments.

1 Setting a virtual environment up

In the previous lab, you had to install a python module named `control`. Here, we will need several modules, including `Gym` and `stable baselines`. One problem is that `stable baselines` does not support versions of `Gym` above 0.21, while `Gym`'s current release version is > 0.26 . If this is your only project, this is not a problem, and when you install `stable baselines`, it will automatically downgrade `Gym`. However, if you have another project on the side which uses `Gym > 0.21`, you will start getting trouble. You would then need to reinstall the correct version of `Gym` each time. The solution to this problem is to create a virtual environment where your python interpreter lives (and finds libraries). You will install all needed modules inside this virtual environment, and this has no effect elsewhere on your computer. You can have similar problems when you want to change versions of the Python interpreter. There are many solutions possible for this problem, from generic tools for package and environment management like `Anaconda`, to more specialized tools like `Poetry`, `pipenv`, `virtualenv` etc.

Update: you should now use `gymnasium` instead of `gym`. The code remains mostly unchanged, a few method signatures have changed.

In this lab, I show you the simplest of solutions that uses the built-in tool `venv`. The following instructions should be entered on the command line (Unix terminal or Windows cmd).

Task 1. Create a new virtual environment: `> python3 -m venv .env`. Here, we ask `venv` to create an new virtual environment called `.env`

You could of course specify any location for the virtual environment.

Task 2. Activate the virtual environment:

Unix : `> source .env/bin/activate`

Windows : `> .env\Scripts\activate.bat`

You are now inside your virtual environment. All modules that you may have installed globally on your computer are not available anymore, so you should install everything that you need.

Task 3. Install `stable baselines` using Python's package manager `pip`: `> python3 -m pip install stable-baselines3`

The `stable baselines` module has many dependencies that are useful to us, and they will be installed automatically (e.g., `Gym`, `Numpy` etc.).

Task 4. We will need the `matplotlib` and `tensorboard` modules. Install them.

2 Setting up the Gym environment

A Gym environment is just an object which standardizes the description of Markov Decision Processes, or environments as we usually call them in Reinforcement Learning. Concretely, the agent will observe the environment's state and issue an action. The environment will respond to that action by transitioning states and issuing a reward. Based on the received reward and the new observed state, the agent is able to learn.

A gym-compliant environment has the following structure:

```
import gym

class CustomEnv(gym.Env):
    def __init__(self, *args, **kwargs):
        # Define action and observation space
        self.action_space = gym.spaces.Discrete(3) # example with 3 discrete actions
        self.observation_space = gym.spaces.Box(low=0, high=255, shape=(128, 128, 3), dtype=np.uint8) # example where the agent gets to observe an image 128x128 RGB image

    def step(self, action):
        return state, reward, is_done, {}
    def reset(self):
        # Reset the state of the environment to an initial state
        return state
    def render(self, mode='human', close=False):
        pass
```

- The class that defines the environment should inherit from `gym.Env`
- In the class' constructor (the `__init__` method), you have to specify the action space (in which space are the possible agent actions) and observation space (in which space are the possible agent observations). In the snippet above, the agent is allowed to use 3 categorical actions (say, more, less, equal), and observes at each step a 128x128 RGB image. To see which spaces are compatible with PPO, visit the stable baselines documentation.
- The `step` method describes how the environment transitions from one state to the other, when receiving a command by the agent. It takes as input the agent's action, and should return a tuple (`state`, `reward`, `is_done`, `info_dictionnary`). `is_done` is a boolean that signals that the task is done or not, and you can return whatever you like inside `info_dictionnary` as long as its a dictionnary.
- the `reset` method describes the environment's initial state. This also includes how to revert the environment back to an initial state after it has finished.
- The `render` method is used to display things sequentially, it is not used during training and we will display our objects all at once, so you can simply leave it as is.

Task 5. Convert the forearm model of last lab to an MDP: specify the potential actions, the possible observations, the rewards, as well as the state transition.

Task 6. Implement a gym environment that behaves like the forearm model, and which issues quadratic rewards exactly like in the LQR setting of last time. You will call this class `ForeArmModel`.

Help. You should not make the action space too large to facilitate learning. In practice, look at the values you had in the previous lab to see how to limit the action space.

Help. Think carefully about the stopping condition.

Help. To facilitate learning, you may also want to increase the sampling period to, say, 50 ms.

Task 7. Verify that your environment is compliant with `stable_baselines3` by using its `check_env` function.

If your environment is compliant with stable baseline, then that means you have followed the API's constraint. However, this does not validate the environment yet, because we have not checked whether the state transitions or the rewards are valid.

Task 8. Use the `LQR` function from the previous lab to generate optimal actions which you can plug into the environment and check whether the state transitions and rewards are valid

Help. Your code may look like this:

```
def check_with_lqr(env, lqr):
    state = env.reset()
    x, y, u, costx, costu = lqr(...)
    for nx, _x in enumerate(x[1:]):
        action = u[nx].reshape(-1)
        state, reward, is_done, _ = env.step(action)

        assert # check state transitions
        assert # check rewards
        if is_done:
            break
```

Depending on the precision of data that you have used (e.g., `numpy.float32` or `numpy.float64`), you may find minor discrepancies.

3 Using PPO to find the optimal control law

Now that the environment is set, we can actually train the policy

Task 9. Train the policy using PPO. To do so, use the following snippet:

```
env = ForeArmModel() # construct the environment
model = PPO("MlpPolicy", env, verbose=1, tensorboard_log="./PPO_ForeArm/") #
                                                                    initialize the agent, log and link
                                                                    to tensorboard
model.learn(total_timesteps=1e5) # train the agent
model.save("ppo_forearm") # save the trained agent for reuse
```

Help. To visualize training, you can open tensorboard. In Unix: `> tensorboard --logdir PPO_ForeArm,` and follow the link that is given. It will open a tab in your browser with a monitor that shows training status.

Task 10. Compare the obtained policy with that from the `LQR`.

Help. To see the result of the trained policy, you can use `model.predict(deterministic=True)`. Remember that PPO learns the components of a Gaussian, and selects actions by sampling from that Gaussian. When you want the best prediction possible you want to take the action with highest likelihood i.e., the mean of the gaussian. This is what the deterministic flag does.